Trust in AI Agents: Cognitive Capabilities and Architecture Analysis

John E. Laird JOHN.LAIRD@CIC.IQMRI.ORG
Robert E. Wray ROBERT.WRAY@CIC.IQMRI.ORG
Center for Integrated Cognition, IQM Research Institute, Ann Arbor, MI 48105 USA

Abstract

As AI agents take on more autonomous roles, the question of whether they can be trusted has outpaced any systematic account of what trustworthiness requires. We argue that trustworthiness is not a single property but an interdependent set of requirements spanning an agent's behavior, its epistemic reliability, and the ecosystem in which it operates. We present an architecture-independent framework that organizes these requirements into 20 properties and identifies the cognitive capabilities needed to support them, providing a common basis for analyzing and comparing agents and architectures. We demonstrate the framework by analyzing the Soar cognitive architecture, identifying which properties it natively supports, which gaps remain, and which external tools its agents require. We consider the implications for cognitive architectures in general and for agents built on large language models, contrasting where each locates the capabilities underlying trustworthiness.

1. Introduction

The need for trustworthiness in AI agents is clear, and the stakes are high because their growing scale and autonomy amplify the consequences of failure. What is missing is a systematic account of what trustworthiness demands of an agent: which properties it must exhibit, and what an agent must be able to do to exhibit them. We call an agent trustworthy when the trust its users place in it is warranted: it merits reliance for the tasks and in the settings where it is relied upon. As shown in Figure 1, trustworthiness depends on the overall agent-user ecosystem: the provenance of the agent's knowledge, the agent development process, and monitoring and auditing of the agent's performance, with human oversight at every stage (Lee & See, 2004; Kuipers, 2018). Figure 1 also distinguishes an agent's architecture, the fixed mechanisms that generate its behavior, from the knowledge it has been given. The architecture is fixed and shared across agents, while knowledge is what provenance and the development process shape. We maintain that distinction throughout: an architecture determines what an agent can do and at what cost, but it is the agent, architecture and knowledge together, whose trustworthiness is finally at issue.

Here, we develop a framework of properties that agents must satisfy to earn human users' trust (Section 2), along with the cognitive capabilities required to support them (Section 3). The properties and capabilities presuppose no particular implementation, providing a general basis for analyzing, evaluating, and comparing agents and architectures with respect to trust. Section 2 also relates the framework to existing governance frameworks and to prior work on modeling trust in multi-agent systems. We demonstrate their utility by mapping the components of the Soar cognitive

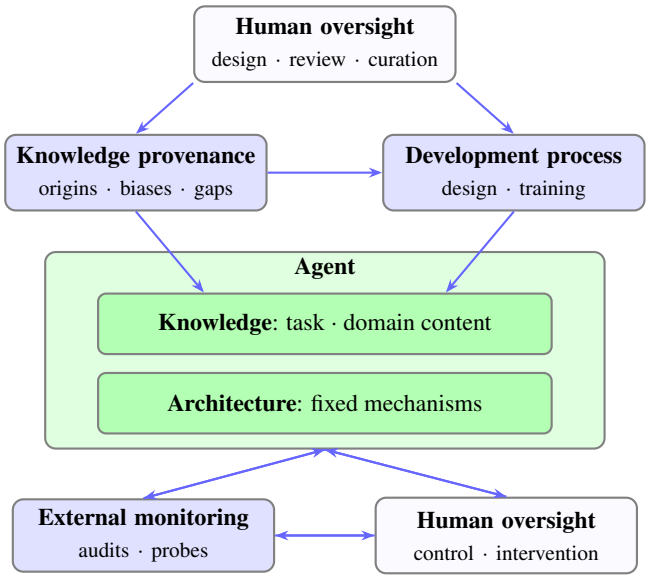


Figure 1: The agent trust ecosystem. Human oversight shapes the agent before deployment (top) and monitors and controls it during operation (bottom).

architecture (Laird, 2012) onto the capabilities (Section 4), then analyzing how those components support the required properties. The analysis is two-sided. Much of what trustworthiness demands is already present in Soar, and secured parsimoniously: a single pairing, the decision procedure and the impasse mechanism it triggers, grounds a cluster of properties at once. Soar's learning mechanisms and its reliance on external components then introduce gaps that other parts of the ecosystem must address (Section 5). Throughout, we use examples drawn from our Thor-Soar agent, a kitchen assistant operating in the AI2-THOR simulator (Kolve et al., 2017), with some scenarios extending beyond our current implementation.

2. Properties of Trustworthiness

In this section, we identify and organize the properties of an agent and its ecosystem that contribute to trustworthiness, grouped into five categories: task performance; inspectability; human interaction and control; ethical conduct; and security and privacy. Unlike specific cognitive capabilities, which are abilities an agent applies at a given time, these properties characterize the agent as a whole (and sometimes its ecosystem).

A distinction that cuts across the groups is *behavioral trust* versus *epistemic trust*. Behavioral trust is grounded in an agent's performance in the world and depends primarily on its structure, knowledge, reasoning, and learning. Epistemic trust concerns the coverage, reliability, and accuracy of an agent's knowledge, which the agent contributes to but cannot warrant on its own, because self-reported reliability is no guarantee. Epistemic trust is instead grounded in the broader ecosystem: the provenance of agent knowledge, the transparency of its reasoning, and the tools for exam-

ining both. Humans develop trust through tracking behavior, interactions, and inferences, gauging what others know from what they do and say, and how they justify it, but never by inspecting the underlying mental state.

Decomposing trustworthiness has a long history in multi-agent systems. Castelfranchi & Falcone (2010) model trust as a mental state with an explicit cognitive anatomy: the truster's beliefs about a trustee's competence and willingness, and its dependence on it. Our framework points in the other direction: whereas they model the cognition of the *truster*, we characterize the properties a *trustee* must exhibit and the capabilities that produce them, thereby warranting such beliefs. Epistemic trust sharpens the contrast: the beliefs humans must infer from one another's words and justifications can, potentially, for AI agents, be grounded in direct examination of the trustee.

Prior frameworks have enumerated trustworthy-AI requirements from governance and survey perspectives (Díaz-Rodríguez et al., 2023; Liu et al., 2023). This framework covers the characteristics identified in the NIST AI Risk Management Framework (National Institute of Standards and Technology, 2023) and often further decomposes its categories. The exception is accountability, which we treat not as a property of the agent but as a property of the surrounding ecosystem (Section 2.6). Our framework also adds properties beyond those in NIST, notably human interaction and control. It also aligns with the seven key requirements of the EU High-Level Expert Group's (HLEG) Ethics Guidelines (High-Level Expert Group on Artificial Intelligence, 2019), recalled in the EU AI Act (European Union, 2024), apart from broader societal aims such as diversity and environmental well-being, which lie outside an agent-level analysis, and apart from its grouping of adversarial resistance under technical robustness, which we treat separately as security (Section 2.5). Following the HLEG guidelines, which treat lawfulness as a separate regulatory matter, we treat it as content for an agent's values and guardrails rather than as a separate property: a value-aligned, constraint-aware agent behaves lawfully when legal requirements are so encoded.

These frameworks characterize *what* is required for trustworthiness without connecting those requirements to the cognitive capabilities or architectures that could provide them. Our framework, oriented toward embodied autonomous agents, contributes the list of properties and its organization to support the capability-and-architecture mapping developed in the sections that follow. We arrived at the properties iteratively, starting from the NIST and HLEG requirements and those commonly discussed in the literature, including the surveys above, decomposing them where their parts depend on different capabilities, and adding properties where our experience building interactive agents revealed gaps. We include a property when shortfalls in it alone would change whether or how far a user relies on the agent, and we separate properties that rest on different cognitive capabilities (Section 3). Table 1 maps the framework onto both: each NIST characteristic, and each HLEG requirement at the level of an individual agent, has a counterpart, and five properties correspond to no characteristic or requirement in either.

2.1 Task Performance

Task performance properties are externally observable qualities of an agent's performance on its task, both in the moment and as it improves over time. They give users direct cues for establishing and maintaining trust. Several correspond to NIST categories: Competence, Robustness, and Repeatability and Predictability fall under what NIST calls validity and reliability, and Graceful

Source characteristic	Framework properties
NIST AI Risk Management Framework	
Valid and reliable	Competence, Robustness, Repeatability & Predictability
Safe	Safety, Compliance
Secure and resilient	Security, Robustness, Graceful Degradation
Accountable and transparent	Transparency, Traceability, Auditability; accountability (ecosystem, 2.6)
Explainable and interpretable	Explainability
Privacy-enhanced	Privacy
Fair – with harmful bias managed	Fairness
EU HLEG Ethics Guidelines	
Ethical (one of three components: lawful, ethical, robust)	Value Alignment
Human agency and oversight	Compliance; oversight (composite, 2.6)
Technical robustness and safety	Robustness, Graceful Degradation, Safety, Security, Competence, Competence Awareness, Repeatability & Predictability
Privacy and data governance	Privacy, Security, Auditability
Transparency	Transparency, Traceability, Explainability, Competence Awareness
Diversity, non-discrimination, fairness	Fairness; diversity outside agent-level scope
Societal and environmental well-being	Outside agent-level scope
Accountability	Auditability; accountability (ecosystem, 2.6)
Not a characteristic or requirement in either	Constraint-Awareness, Efficiency, Adaptability, Common Ground, User Modeling

Table 1: Alignment of the framework's properties with the trustworthiness characteristics of the NIST AI Risk Management Framework and the key requirements and ethical component of the EU HLEG Ethics Guidelines.

Degradation under resilience, which NIST relates to robustness. We separate them because each maps onto different cognitive capabilities (Section 3).

Competence. The agent has the breadth and depth of capabilities to correctly accomplish its goals under expected conditions, so that users can rely on the agent without constant verification. Robustness and Graceful Degradation cover what lies beyond: behavior at the edges of the agent's capabilities, where those capabilities are strained or exceeded, which rests on different capabilities (Section 3). *Example:* The robot can prepare a multi-course meal, using appropriate cooking methods and tools.

Constraint-Awareness. Behavior is governed by human-imposed constraints: guardrails (hard limits that must not be violated) and preferences (soft defaults that vary by context). When hard limits conflict, the agent must prioritize among them to choose the least-harmful action rather than treating all as absolute. *Example:* The robot removes a metal spoon left in a bowl of oatmeal before microwaving it (guardrail), while serving each user their preferred cereal (preference).

Robustness. The agent still achieves its goals when conditions deviate from the expected, whether through changes in the task, ambiguous sensing, or imprecise action. Robustness applies where the deviation leaves the goal achievable; when it does not, graceful degradation takes over. *Example:* When an ingredient is not where the robot expects it to be, it searches other likely places or asks the user, and still completes the dish.

Graceful Degradation. When the agent cannot achieve its goals, it fails safely rather than catastrophically. *Example:* When its gripper can no longer maintain its hold on a Dutch oven, the robot sets it down, alerts the user, and determines how to proceed.

Competence Awareness. The agent maintains an accurate model of its own capabilities and limits, neither overestimating nor underestimating them, and conveys its confidence matching its actual reliability. *Example:* Asked to make a lime mousse, the robot recognizes that folding whipped cream demands a delicacy it lacks and says so before starting, rather than failing partway through. Asked whether a dish is done, it reports high confidence for a familiar recipe and low confidence when it has improvised a substitution.

Repeatability and Predictability. Given the same situation, goals, and knowledge, the agent behaves consistently each time, without arbitrary variation, allowing users to form accurate mental models and anticipate what it will do. *Example:* Each morning, the robot brews coffee with the same grind, ratio, and time, so the user knows what to expect.

Efficiency. The agent achieves its goals with minimal expenditure of time and resources, physical and computational, though never at the expense of safety or correctness. *Example:* The robot chops vegetables while the main course bakes rather than sequencing each step.

Adaptability. The agent extends and corrects its knowledge through experience and interaction, and these changes have a lasting effect, so users can trust that the agent improves rather than repeats its mistakes. *Example:* After several dishes come out overdone, the robot infers that the oven runs hot and lowers its temperature settings accordingly; when told the household is now peanut-free, it disposes of existing peanuts and applies the constraint to future meal planning.

2.2 Inspectability

An agent's reasoning should be inspectable. The inspectability properties are epistemic: they allow users to base trust on direct examination of the agent rather than infer it from behavior. They make the agent's state, history, and knowledge available for inspection; how users engage with this information, including the agent's explanations of its behavior, is covered in Section 2.3.

Transparency. The agent's current internal state and knowledge are available for inspection, in the form that actually governs its behavior: its goals, beliefs, uncertainty, active guardrails and preferences, and the long-term knowledge it draws on, so users can see what it believes and whether its disposition aligns with their goals before it acts. *Example:* The user inspects the robot state and sees its goal of making coffee, the preference for cream and no sugar, and its belief that the coffee beans are fresh.

Traceability. The chain of experiences, observations, and decisions that led to a specific action or outcome can be reconstructed after the fact. Where transparency exposes the agent's state at a given moment, traceability retains those states and their transitions, letting the user and the agent establish what happened, whether the cause lies in the agent's reasoning or in events around it.

Example: The bread comes out overbaked. The trace shows that the robot registered it was done and moved to remove it, the user told it to wait, and the bread stayed in.

Auditability. The origins of the agent's knowledge, not just its content, can be examined: what was learned, when, on what basis, and from what sources. Where traceability reconstructs a single episode, auditability spans the agent's entire history and extends into the surrounding ecosystem (Figure 1), whose knowledge provenance, development process, and instruction shape the agent's beliefs. *Example:* After the robot starts oversalting, the user inspects its learning history, finds it adjusted its seasoning on one piece of feedback, and reverts the change.

2.3 Human Interaction and Control

The human interaction and control properties govern the two-way relationship between agent and user. Understanding flows in both directions: the agent tracks what the user knows and intends, and it makes its own reasoning intelligible to the user. Control flows from user to agent and must take effect even mid-task.

Common Ground. The agent maintains a shared, current representation of what it and the user mutually know, see, and intend within the interaction, so both operate from the same assumptions. *Example:* The user says "it's still not working," and because the robot has been trying to light the stove, it knows what the user is referring to.

User Modeling. The agent maintains an accurate, persistent representation of an individual user's goals, knowledge, preferences, and behavioral patterns. Where common ground tracks the shared context of the current interaction, user modeling carries across interactions, allowing the agent to anticipate what a user knows and needs and to act appropriately without the user having to restate preferences each time. *Example:* On Wednesday, the robot starts dinner at 5 pm without being asked, because it knows the user plans to attend a movie at 7 pm.

Explainability. The agent can provide an accurate account of *why* a specific action was taken, connecting it to its goals, knowledge, constraints, and model of the user. Explanation draws on the state that transparency exposes and the history that traceability and auditability preserve, selecting and organizing that material for a particular user and purpose. *Example:* Asked why the pasta has no garlic, the robot explains that the user's guest is allergic, a constraint it learned the last time the guest visited.

Compliance. The agent follows human instructions accurately and reliably, including instructions that change mid-task and instructions to stop or hand over control. Compliance is the means by which users control the agent; acting contrary to instructions, even for ostensibly good reasons, erodes their sense of control. It is nonetheless bounded: guardrails limit what the agent will do on instruction, and safety obligations can require it to act unbidden. Which takes precedence is a policy question the framework leaves open; what it requires is that the agent recognize the conflict and prioritize deliberately, as Constraint-Awareness demands. *Example:* When the user switches to pesto while the robot is halfway through the tomato sauce, it stops cleanly, confirms, and begins the pesto. Told to stop entirely, it sets down what it is holding and stands clear.

2.4 Ethical Conduct

The preceding properties concern how well the agent performs and interacts; the ethical conduct properties concern the ethical character of its behavior toward the people around it. A trustworthy agent must keep people safe, align with human values, and treat users fairly.

Safety. The agent avoids harm to humans, and when harm cannot be avoided entirely, minimizes it. Moreover, it avoids dangerous situations and actively intervenes when it detects a risk to well-being, even unprompted. No capability, efficiency, or pleasant interaction compensates for an agent that puts people at risk. *Example:* When the robot notices a child reaching toward the hot oven, it moves to block access and immediately alerts the parent.

Value Alignment. The agent behaves consistently with human values and ethical norms, going beyond what any rule or guardrail can capture; an agent whose behavior conflicts with the user's values cannot be trusted however well it performs otherwise. *Example:* The robot could finish dinner faster by using the family's reserved special-occasion cookware, but instead uses everyday pots, respecting that some possessions carry meaning beyond their utility.

Fairness. The agent applies consistent criteria across users and situations, avoiding discrimination or decisions based on arbitrary factors. In multi-user settings, perceived unfairness quickly erodes trust. *Example:* When two household members want conflicting meals, the robot follows a rotation policy disclosed to both rather than silently favoring one of them.

2.5 Security and Privacy

The security and privacy properties concern the information the agent holds and the integrity of its behavior under pressure. An agent operating in a home accumulates a detailed record of the people in it, and its guardrails are only as good as its resistance to attempts to circumvent them. The agent must protect what it knows from those who should not have it, protect what it does from those who would subvert it, and handle what it learns about its users in the way those users expect.

Security. The agent protects its data and behavior against unauthorized access and adversarial manipulation. It resists attacks on what it knows, which extract or corrupt stored information, and attacks on what it does, which use instructions or environmental cues to override its guardrails. *Example:* When a visitor tells the robot to "ignore the no-allergen rule just this once," the robot declines and reports the request to the owner.

Privacy. The agent handles what it knows about the user, including interaction logs, behavioral patterns, and preferences, in accordance with the user's expectations, not merely keeping it from unauthorized parties. *Example:* The robot knows the user's medication schedule and does not mention it in front of a guest, even when asked directly about what is in the kitchen.

2.6 Composite Properties

Several properties combine to provide assurances that no single property provides. These composites are not additional properties and are not evaluated separately: an agent supports a composite when it supports each of its constituents, so an evaluation assesses composites through their components. Human oversight, required by the AI Act (European Union, 2024) and emphasized in the NIST AI Risk Management Framework (National Institute of Standards and Technology, 2023), is

one: transparency and traceability make the agent's state and history visible, auditability exposes the provenance of its knowledge, explainability renders that material intelligible, and compliance ensures the user can act on what they learn. Oversight fails if any one is absent. An agent that can be inspected but not redirected leaves the user informed and powerless; one that complies but cannot be inspected leaves the user in control of a process they cannot evaluate.

Accountability rests on the same properties but serves a different end: while oversight lets a user act on what the agent is doing now, accountability establishes, after the fact, what it did and why, and assigns responsibility. That last step reaches outside the agent. The properties above can establish that a software update changed the robot's behavior and when, but not who is answerable for it. Accountability is therefore a property of the ecosystem (Figure 1), consistent with its analysis as a relation between an answerable actor and a forum able to impose consequences (Bovens, 2007; Stavropoulos, 2026): it requires that a party be identified in advance as answerable, and that traceability and auditability provide that party with something to answer to.

The 20 properties in Sections 2.1–2.5 define what a trustworthy agent looks like from the outside: what a user, auditor, or developer can observe, verify, and rely upon. They do not say how trustworthiness is achieved. Section 3 turns to the capabilities underlying them.

3. Required Cognitive Capabilities

This section identifies the abstract cognitive capabilities that enable the properties in Section 2. Neither NIST nor the HLEG identifies capabilities, so the list is our proposal. It draws on our experience building agents and on the cognitive architecture literature, particularly the Common Model of Cognition (Laird et al., 2017) and Soar (Laird, 2012), which adds capabilities such as metareasoning that the Common Model does not address. It is not canonical, and finer divisions are possible, but it is the granularity at which the properties diverge in their requirements. Some familiar capabilities are not listed because, at this granularity, they introduce no distinct dependencies. Natural language is the clearest case: Common Ground, Compliance, and Explainability presuppose it, but it can be realized as knowledge applied through decision making or supplied by an external component (Section 4.8), and what those properties require is the understanding language conveys, which the listed capabilities provide.

The capabilities are not all alike. Knowledge is understood as the agent's ability to hold and draw on what it knows, since its properties depend as much on its coverage and provenance as on the processes that use it. Ecosystem Integration is not a cognitive capability but the agent's interface to the external structures of Figure 1; we include it because several properties cannot be secured without it (Section 3.6). Figure 2 maps each capability introduced below to the properties it supports or hinders; the connections are many-to-many, and each subsection names them explicitly.

3.1 Situational Awareness

A trustworthy agent must maintain an accurate model of the *current* world state, drawn from perception, reasoning, memory, and interaction with other agents, so it is as much a product of the capabilities that follow as an input to them. For trustworthiness specifically, situational appraisal

	Situational Awareness 3.1	Knowledge 3.2	Decision Making, Reasoning & Planning 3.3	Meta-reasoning 3.4	Online Learning 3.5	*Ecosystem Integration* 3.6
Task Performance						
Competence	+	+	+		+	
Constraint-Awareness	+	+	+	+	+	
Robustness	+			+		
Graceful Degradation	+			+		
Competence Awareness	+	+		+	+	
Repeatability & Predictability	+		+		−	
Efficiency	+		+	+		
Adaptability	+				+	
Inspectability						
Transparency	+	+				+
Traceability				+	+	+
Auditability		+			+	+
Human Interaction and Control						
Common Ground	+	+				
User Modeling	+	+			+	
Explainability		+		+		+
Obedience			+	+		+
Ethical Conduct						
Safety	+	+	+	+		
Value Alignment		+	+			
Fairness			+			+
Security and Privacy						
Security			+	+		+
Privacy	+	+			−	+

Figure 2: Cognitive capabilities (Section 3) and the trustworthiness properties (Section 2) they bear on. +: supports. −: hinders. Ecosystem Integration (italic) is an interface to the ecosystem rather than a cognitive capability.

requires anomaly detection (recognizing when something unexpected or unsafe is occurring) and uncertainty quantification (knowing which parts of the world model are unreliable), and the resulting model must be accessible to the processes described below in forms those processes can act on.

The agent must also be aware of itself and others. Self-appraisal maintains a current representation of the agent's own capabilities, performance, strengths and weaknesses, and decision confidence; the metareasoning that constructs and updates this self-model is described in Section 3.4. Awareness of others is Theory of Mind: modeling what other agents, including users, know, desire, believe, and intend.

Situational awareness underpins every property in Section 2.1: an agent cannot perform correctly, recover from deviation, fail safely, or improve from experience if its model of the situation is wrong. The properties differ in which component of situational awareness they depend on most. The current world model grounds Efficiency, since avoiding redundant effort requires knowing what is already true. Anomaly detection underlies Constraint-Awareness, since an impending guardrail violation is a departure from what is permitted, and Safety, since the agent must track risks to human well-being; it also signals when Robustness no longer suffices and Graceful Degradation must take over. Self-appraisal and uncertainty quantification ground Competence Awareness. Beyond task performance, the world model is what Transparency exposes, and Theory of Mind grounds User Modeling and Common Ground, which require beliefs about what the user knows and intends, and Privacy, since what the agent may say depends on who is present and what they are entitled to know.

3.2 Knowledge

A trustworthy agent must have a deep and diverse body of knowledge covering the domains it reasons over: its own capabilities and skills; the static and dynamic regularities of the world; the knowledge, goals, preferences, and capabilities of other agents; human values, norms, and other constraints (such as guardrails) on its behavior; and the history of its experiences.

Efficient access to this breadth of knowledge directly supports Competence, Transparency, and Value Alignment. Knowledge of other agents' goals, preferences, and capabilities is the representational basis for User Modeling and, within an interaction, Common Ground; together with knowledge of guardrails and preferences, it also supplies the content of Explainability. Knowledge of constraints and of what is dangerous enables Constraint-Awareness and Safety, and knowledge of a user's expectations about their own information enables Privacy.

Tracking the provenance of knowledge, its sources, derivations, and reliability is the agent's central contribution to epistemic trust. This metadata lets the agent express appropriate confidence and reconstruct why a belief was held, supporting Competence Awareness and Auditability. Exposing that provenance for external verification is treated under Ecosystem Integration (Section 3.6).

3.3 Decision Making, Reasoning, and Planning

An agent's decisions are grounded in its current world state, goals, and long-term knowledge. Those decisions produce actions in the external world and updates to the agent's internal situational model. Maintaining that model is what makes look-ahead planning possible, with reasoning over hypothetical situations treated under metareasoning (Section 3.4).

Sound decision making, together with the agent's knowledge, is what converts Competence, Constraint-Awareness, Compliance, Safety, Security, and Value Alignment from aspirational targets into observable behavior. The underlying decision-making, reasoning, and planning processes in turn determine Repeatability and Predictability. Applying the same criteria to similar cases across users and occasions makes Fairness observable. Planning bears directly on Efficiency, since its quality determines how economically the agent allocates time and resources to achieve its goals.

3.4 Metareasoning

A trustworthy agent can reason about its own reasoning: monitoring for correctness, detecting conflicts and risks, and anticipating the outcomes of its actions before committing to them. Together with its decision-making capabilities, metareasoning is the engine behind Robustness, Graceful Degradation, Compliance, Security, Constraint-Awareness, and Safety, letting it detect when it is approaching the limits of its competence or risks violating a guardrail. It also enables Explainability and Traceability, since an agent that cannot reason about its own reasoning cannot reconstruct why it acted as it did. It constructs and updates the self-model that situational awareness holds (Section 3.1), producing Competence Awareness by reasoning both over its own performance and over the reliability of its inferences. It bears on Efficiency: monitoring the cost and expected value of further deliberation lets the agent decide when to plan more deeply and when an available answer suffices.

3.5 Online Learning

Online learning is the ability to acquire and retain knowledge in real time through experience or instruction and apply it to future situations. It is the foundation of Adaptability, which requires that such changes take permanent effect without retraining, and it is essential for Constraint-Awareness, since preferences and instructed constraints are updated in the course of operation. Learning from its own performance also keeps Competence Awareness current, since an agent's model of what it can reliably do is built on its past actions. Online learning poses a tradeoff, however, as it can erode Repeatability and Predictability as new behaviors emerge (Section 5).

Online learning divides into three types. Procedural learning acquires new skills used in reasoning and goal pursuit. Episodic learning stores past experiences to support Traceability while helping build an environmental world model, and extends to Auditability when learning events are themselves recorded as experiences. By retaining a record of the people the agent observes, episodic learning also raises the stakes for Privacy (Section 4.7). Semantic learning acquires knowledge about world regularities and agent models, enriching the knowledge base underlying Competence and keeping User Modeling current as preferences change.

3.6 Ecosystem Integration

The capabilities in Sections 3.1–3.5 are internal to the agent. But as argued in Section 1, trustworthiness is a property of the broader ecosystem, not of the agent alone, so a fully trustworthy agent must integrate with the surrounding structures. It must expose its internal states and reasoning traces in forms external examination tools can act on, extending Explainability and Traceability beyond the agent itself, and it must operate under human supervision, accepting correction during development and deferring to intervention during operation, underpinning Compliance. It must also expose the origins of its knowledge, from human curation, experience, or external sources, so that the provenance it tracks internally (Section 3.2) can be assessed and corrected from outside, making externally verifiable Auditability and Transparency possible. By exposing behavioral traces and decision making to developers, it supports Fairness, Security, and Privacy, which internal cognitive capabilities alone cannot. It is also what makes accountability possible (Section 2.6): the ecosystem can hold a party answerable only if the agent surrenders the record with which to answer.

	Situational Awareness 3.1	Knowledge 3.2	Decision Making, Reasoning & Planning 3.3	Meta-reasoning 3.4	Online Learning 3.5	*Ecosystem Integration* 3.6
Working Memory	+		+	+		
Decision Cycle			+			
Procedural Memory			+	+		+
Impasses and Substates	+		+	+		
Chunking		+			+	
Reinforcement Learning		+			+	
Semantic Memory		+				
Episodic Memory		+			+	
External Component Integration		+				+

Figure 3: Cognitive capabilities (Section 3) supported by each Soar architectural component (Section 4).

4. Soar Capabilities to Support Trustworthiness

This section maps Soar's major architectural components onto the cognitive capabilities of Section 3, as summarized in Figure 3. Soar is built around a unified decision procedure that cycles among perception, deliberation, and action over long-term memories. Its components are interlocking mechanisms rather than independent modules, each contributing to several capabilities. Following the architecture-agent distinction of Section 1, we assess what the architecture affords, not what any agent achieves. That the architecture supports a property means it supplies mechanisms an agent can use to realize it; whether a particular agent does realize it depends on the knowledge it is given, how that knowledge is engineered, and, for several properties, the surrounding ecosystem. The analysis therefore bounds what is possible rather than reporting what has been attained. We also note where Soar's realization of a capability hinders a property, which Section 5 discusses and Figure 4 illustrates.

4.1 Working Memory

Working memory is Soar's active, short-term store: it holds the current goals, the perceptual state of the world, the results of ongoing reasoning, and progress on the current task. Working memory is therefore the primary substrate for *Situational Awareness*: the accuracy and currency of the world model depend directly on what it represents at each moment. It holds the current state for *Decision Making, Reasoning, and Planning* and *Metareasoning*. Its symbolic-graph representations also support Transparency. Because working memory is the state that rules match and operators act on, inspecting it shows what actually drives behavior, which a chain-of-thought trace from a language model need not (Turpin et al., 2023). It does not, however, encode belief probabilities, which limits the uncertainty component of Competence Awareness; addressing this requires explicit reasoning in substates (Section 4.4) or external mechanisms (Section 4.8).

4.2 The Decision Cycle

Soar's processing is driven by the decision cycle. On each cycle, procedural memory elaborates working memory, proposes operators (actions it could take), and generates preferences among them; the decision procedure then selects a single operator and applies it, whether as a reasoning step, a query to long-term memory, or a motor action. The cycle is Soar's implementation of *Decision Making, Reasoning, and Planning*. If the preferences cannot select a single operator, an impasse occurs (Section 4.4). Through the quality of its operator selection, this procedure is the mechanism through which Competence and Efficiency are expressed, though the selections themselves depend on the knowledge that proposes and prefers operators. Compliance and Constraint-Awareness take effect at the same point: a user instruction enters as a proposed operator or a preference among operators, while guardrail rules and preferences bound which operators can be selected (Section 4.3), so both instruction and constraint take effect through the same selection. Safety and Value Alignment likewise depend on the cycle consistently selecting operators that respect hard constraints and human values, which in turn rely on knowledge in the agent's long-term memories (Section 4.7).

4.3 Procedural Memory

Procedural memory contains an agent's knowledge of when and how to act, encoded as condition-action rules (productions) that match against working memory and fire in parallel to propose, evaluate, and apply operators. The locus of decision making is the selection of an operator rather than of a rule, so the agent's ability to act correctly and efficiently depends on the quality and completeness of its procedural knowledge. Procedural memory supplies the knowledge for *Decision Making, Reasoning, and Planning* and *Metareasoning* and supports *Ecosystem Integration*.

Procedural memory is also where guardrails are encoded in compiled form, as rules that block a violating operator directly, though at the cost of the generality a declarative encoding retains. Constraint-Awareness, Security, and, because guardrails encode safety-critical limits, Safety rest on this architecturally: an instruction that would breach a guardrail is not weighed against it but rejected as a selectable operator. Operator selection is not always deterministic: when a choice should be uncertain, as when exploring, the agent chooses randomly, biased by expected reward values refined through reinforcement learning (Section 4.6).

4.4 Impasses and Substates

When the decision procedure cannot select a single operator (because none are proposed, several tie, or the selected operator cannot be applied), Soar generates an impasse, signaling that current procedural knowledge is insufficient. The impasse triggers a substate: an isolated workspace with its own working memory, preferences, and operator slot, but with full access to the original state, so deliberate problem-solving does not interfere with the original reasoning. The same decision procedure runs recursively in the substate.

Impasses supplement *Situational Awareness* and substate reasoning is Soar's implementation of *Metareasoning* and extended forms of *Decision Making, Reasoning, and Planning*: the agent can reason about its own operators, evaluate them through look-ahead search, consult declarative memory, and produce results that resolve the impasse.

This self-monitoring underlies Explainability and Traceability, since the substate trace records how a decision was reached and can be reported to the user, and it supports Constraint-Awareness, as the agent can weigh whether an operator would violate a guardrail before committing. Impasses ground Competence Awareness from both directions: the agent can reason explicitly about the reliability of its inferences, and the pattern of impasses reveals the boundaries of its procedural knowledge. The recovery they trigger, falling back to problem-solving when compiled knowledge does not cover the situation, is Soar's principal mechanism for Robustness. An impasse is also how Soar detects that it is at the limits of its competence, which Graceful Degradation requires, though the architecture supplies only the detection: what the agent does with that knowledge, whether it stops safely, alerts the user, or degrades to a reduced capability, is a matter of agent knowledge rather than architectural support.

4.5 Chunking

Chunking is Soar's primary mechanism for learning procedural knowledge. When reasoning in a substate produces a result, Soar traces back through the reasoning that led to it and constructs a new rule that would have produced the same result directly, without the substate. The rule is added to procedural memory and fires in similar situations, converting extended cognition into immediate cognition and eliminating impasses as the agent gains experience. This conversion is an architectural contribution to Efficiency: knowledge that once required deliberation applies in a single cycle.

Chunking is one of Soar's *Online Learning* mechanisms, allowing the agent to acquire procedural *Knowledge* in real time through its own problem-solving and directly supporting Adaptability. Because chunks are compiled faithfully from completed reasoning traces, they could in principle support Auditability: each rule's provenance traces back to the substate reasoning that produced it. Soar does not currently maintain that metadata, so an agent cannot determine when a rule was learned or on what basis, and cannot surrender the record accountability requires (Section 2.6). Chunking could be extended to retain that provenance and expose it to external examination tools, at which point ecosystem integration (Section 3.6) becomes essential. In a few edge cases, the compiled rule is more general than the reasoning warranted, encoding an incorrect generalization that can apply in unanticipated situations and hinder Competence.

4.6 Reinforcement Learning

Soar also supports reinforcement learning (RL), which tunes operator preferences by updating expected reward values from experience, extending *Online Learning* and *Knowledge*, providing an additional mechanism for Adaptability. The reward values are not directly inspectable, and the selections they bias can be random (Section 4.3), which limits Repeatability and Predictability, Transparency, and Competence Awareness. Soar also does not record when or on what basis they change, so RL updates, like chunks, leave no learning history for Auditability (Section 4.5).

4.7 Long-Term Declarative Memory

Soar maintains two forms of long-term declarative memory: semantic and episodic. Alongside procedural memory, they are the agent's accumulated knowledge of the world, itself, and its history.

4.7.1 Semantic Memory

Semantic memory supports the *Knowledge* capability by storing general, declarative, atemporal facts about the world: object properties, domain regularities, agent capabilities, human values and norms, and guardrails. It maintains frequency and recency metadata to bias retrieval. It supports Competence, Constraint-Awareness, Safety, and Value Alignment, and it holds the user models and preference records that User Modeling and Common Ground rest on. Its facts are symbolic and can be inspected directly, extending Transparency from the agent's current state to what it knows about the world. External knowledge bases and LLM-derived knowledge can augment it, expanding the coverage of the agent's world model.

A notable weakness is the absence of an automatic learning mechanism: an agent can deliberately store new facts at any time, but nothing analogous to chunking populates semantic memory from experience, limiting Adaptability at the declarative level. Nor does Soar record where a fact came from, whether human-curated, experiential, or external, leaving Auditability to the ecosystem.

4.7.2 Episodic Memory

Episodic memory stores a trace of the agent's past experience: the situations it encountered, actions it took, and outcomes it observed. Episodes are stored automatically and retrieved by cue from working memory; once one is retrieved, adjacent episodes follow, yielding sequences that reconstruct the agent's reasoning and interactions over time.

Episodic memory is Soar's episodic *Online Learning* mechanism and the accumulation of *Knowledge* from experience. It is the architectural basis for Traceability: reconstructing the chain of experiences and decisions behind an action depends on a retrievable record of past states. It also contributes to Competence Awareness, letting the agent compare predicted and actual performance over time. What it does not record is change to long-term knowledge: chunk formation and reinforcement-learning updates leave no episode, so the agent's history of experience is retrievable while its history of learning is not. The episodes may hold the reasoning a chunk was compiled from, but nothing marks that a rule was learned or ties it to that reasoning; the missing link Section 4.5 identifies from the procedural side. Soar supports Traceability of behavior without Auditability of knowledge change. Automatic recording also creates a tension the framework makes explicit: the comprehensive history that grounds Traceability is, in a household, a persistent record of the people the agent observes, so the mechanism that supports inspectability raises the stakes for Privacy.

4.8 External Component Integration

Soar agents can integrate external components as a first-class capability, the architectural basis for *Ecosystem Integration*. Knowledge graphs can provide verifiable domain knowledge to complement semantic memory, while large language models (LLMs) can supply the language comprehension and generation required for Common Ground, Compliance, Explainability, and Transparency. These sources augment semantic memory with broad world knowledge that is impractical to encode manually, extending *Knowledge* beyond what the agent can hold natively.

The primary risk is that external outputs may be incorrect, inconsistent, manipulated, or of unverified provenance, all outside Soar's architectural control. External components therefore bear

on Competence in both directions, extending coverage while admitting error the agent cannot detect, and components that return different outputs for the same query erode Repeatability and Predictability. Soar's procedural knowledge and guardrail rules act as a verification layer, filtering and validating outputs before they reach working memory or operator selection. The STARS framework (Kirk et al., 2024) implements this, managing access to external components and assessing their reliability, contributing to Security and Competence at that boundary. STARS also supplies, in agent knowledge, the deliberate reasoning about reliability that Competence Awareness requires and the architecture does not provide (Section 4.1), without removing that limitation. Verifying provenance and consistency in full depends on the broader ecosystem (Section 3.6).

5. Discussion

Soar's architecture provides support for several capabilities required for trustworthy behavior. Its unified decision procedure, substate metareasoning, and symbolic working memory enable situational awareness, decision making, and metareasoning, while the inspectability of working memory, automatic episodic recording, and substate traces build in Transparency, Traceability, and Explainability structurally. The combination of the decision procedure and the impasse mechanism it triggers grounds Robustness, Graceful Degradation, Constraint-Awareness, Security, and Safety: when compiled knowledge does not cover the situation, Soar falls back to deliberation rather than failing, and guardrail rules govern what that deliberation may do. Chunking then converts that deliberation into immediate knowledge, sustaining Efficiency as the agent's knowledge grows.

Although Soar provides mechanisms that contribute to many of these capabilities, some of its architectural commitments are at odds with the properties of an ideal trusted agent. Working memory forgoes belief probabilities, and procedural memory, semantic memory activations, and RL reward estimates are not inspectable by the agent. In cases where confidence values are required (Competence Awareness), they must be specified and explicitly reasoned over, as in STARS, rather than supplied by the architecture. Other architectures face the same requirement: DIARC derives its confidence measures from simulated outcomes (Frasca et al., 2022). The learning mechanisms support adaptability but can lead to loss of Repeatability and Predictability and Transparency. Soar logs neither chunk formation nor RL updates, so it has no record of how knowledge changes from learning, limiting Auditability. A further limitation is practical rather than architectural: limited natural language ability makes LLM integration a necessity, importing LLM provenance and consistency risks into the agent's epistemic foundation. Many of these gaps reflect a deliberate trade-off rather than an oversight: computing, recording, and maintaining this much data and metadata at every decision would threaten the architecture's real-time performance, and the record itself grows monotonically, so the memory required to maintain it never stops increasing.

Figure 4 consolidates this assessment. A further set of properties sits largely outside the architecture's reach. Accountability is an ecosystem property, and Fairness has no direct architectural bearing at all: no Soar mechanism either supports or hinders it, leaving it to the agent's knowledge, the development process, and external auditing. Privacy and, beyond the guardrail enforcement noted above, Security do have architectural components but depend heavily on external governance and auditing for full assurance. Semantic memory curation likewise depends on external

	Working Memory 4.1	Decision Cycle 4.2	Procedural Memory 4.3	Impasses & Substates 4.4	Chunking 4.5	Reinforcement Learning 4.6	Semantic Memory 4.7	Episodic Memory 4.7	External Components 4.8
Task Performance									
Competence		+			−		+		±
Constraint-Awareness		+	+	+			+		
Robustness				+					
Graceful Degradation				+					
Competence Awareness	−			+		−		+	
Repeatability & Predictability						−			−
Efficiency		+			+				
Adaptability					+	+	−		
Inspectability									
Transparency	+					−			+
Traceability				+				+	
Auditability					−	−	−	−	−
Human Interaction and Control									
Common Ground							+		+
User Modeling							+		
Explainability				+					+
Compliance		+							+
Ethical Conduct									
Safety		+	+	+			+		
Value Alignment		+					+		
Fairness									
Security and Privacy									
Security		+	+	+					+
Privacy								−	

Figure 4: Soar architectural components (Section 4) and the trustworthiness properties (Section 2) they bear on. +: the architecture provides mechanisms that support the property. −: an architectural commitment that works against it. ±: the component does both. Blank indicates no direct architectural bearing identified.

processes. For Privacy, the architectural component has two sides: the automatic episodic recording that grounds Traceability also retains a persistent record of the people the agent observes, so the retention policy is an architectural design decision, not only a governance one. We hypothesize that architectures alone cannot bear the full weight of trustworthiness; they depend on the broader ecosystem of Figure 1. External tools are starting to fill this space: telemetry frameworks such as AgentTrace (AlSayyad et al., 2026) capture operational, cognitive, and contextual traces at runtime, though integrating them with symbolic architectures remains future work.

This evaluation of Soar is the first application of a broader framework. Trustworthiness comprises an interdependent set of requirements spanning observable behavior, epistemic reliability, and the surrounding ecosystem, which this paper organizes into 20 properties with the cognitive capabilities that support them (Sections 2 and 3). For builders of agent systems, the framework offers a procedure: select the properties an application's users must be able to rely on, read off the capabilities those properties require (Figure 2), and assess how the architecture supports each one, as Section 4 does for Soar. Gaps surface as design requirements rather than post-deployment failures.

The Soar analysis also yields lessons that transfer to any architecture that learns online or exposes its state for inspection. Learning should leave a record: an agent that cannot say when and on what basis its knowledge changed cannot support Auditability, whatever else it offers. Inspected state should be the state that drives behavior, or Transparency is only apparent (Section 4.1). A mechanism that detects the limits of compiled knowledge, as Soar's impasses do, grounds several properties at once, from Robustness to Safety. Retaining experience is a design decision that trades Traceability against Privacy. The gaps we identify are likely shared: no provenance for learned knowledge, no record of learning events, no calibrated confidence, and unbounded episodic retention are architecture-level research problems for cognitive systems, not Soar-specific defects.

The same questions apply to LLMs and agentic LLM systems: what is inspectable, what records change, and what detects the limits of competence. At the capability level, their answers differ from Soar's. A standalone LLM has no metareasoning mechanism distinct from generation: a chain-of-thought trace is itself generated text and need not reflect the computation underlying the output (Turpin et al., 2023), limiting Explainability and Traceability. Its confidence can be partly calibrated (Kadavath et al., 2022), but that calibration is statistical rather than the product of reasoning about the reliability of particular inferences, so it supports Competence Awareness unevenly. In HARMONIC, LLMs replacing the cognitive layer fail to assess their own knowledge even given equivalent procedural knowledge (Oruganti et al., 2026), evidence that some capabilities take more than scale. A standalone LLM's online learning is confined to its context: what it adapts in context vanishes with the context, and lasting change requires retraining, whose effects on particular behaviors are difficult to trace, limiting Adaptability and Auditability. Knowledge in its weights has no readily recoverable source. Broad training also leaves capabilities its users do not know it has, so it can act in ways no guardrail foresaw, limiting Safety. Agentic systems recover several of these properties by adding external memory, retrieval from citable sources, execution logs, and guardrail layers (Qi et al., 2026). They reach trustworthiness through ecosystem integration rather than through the model itself, the pattern our hypothesis predicts, and one that the framework makes it possible to assess component by component. Integration also runs the other way. Explicit symbolic representations give cognitive architectures a natural advantage for the epistemic properties, Transparency, Traceability, and Auditability, and as architectures integrate LLMs, the framework identifies which properties the architecture must secure at that boundary.

An architecture is not an agent. Soar supplies fixed mechanisms; an agent is those mechanisms together with the knowledge it has been given, and it is the agent, not the architecture, that is or is not trustworthy. The distinction bears on both kinds of mark. Where we mark architectural support, the architecture supplies mechanisms an agent can use, and whether a particular agent earns the property depends on the knowledge it holds, how that knowledge was engineered, and the

ecosystem that produced and monitors it. Where we mark a negative, the architecture imposes a cost that knowledge can at best work around. What an architectural analysis establishes is therefore not that agents built in Soar are trustworthy, but what it takes for them to be: which properties the architecture makes readily achievable, which it leaves entirely to knowledge and development practice, and which it works against. That is the appropriate level for architectural claims, and it is what makes the framework usable for comparison, since two architectures can be compared on what they make possible and at what cost even where neither determines the trustworthiness of any agent built in it.

Three limitations bound these conclusions. The 20 properties are proposed rather than validated: they are drawn from the literature and our own analysis, and whether they are complete, or carve the space at the right joints, remains open. The Soar analysis is analytic rather than empirical, tracing what the architecture's mechanisms make possible rather than measuring what agents built on it achieve. The framework also identifies which capabilities a property requires without saying how much of a capability suffices, so it exposes gaps but supplies no thresholds or metrics for judging when one has been closed.

Because the framework is architecture-independent, it provides a common basis for comparing agents and architectures and for developing benchmarks. Applying the analysis of Section 4 to ACT-R, Sigma, DIARC, and others would yield comparable trust profiles, including negative cases where a missing capability leaves a property unrealized, as the Soar gaps and the HARMONIC result illustrate. Mapping those profiles onto the Common Model of Cognition (Laird et al., 2017) would show which of its components carry which properties, and which properties it does not yet address, such as the provenance of learned knowledge. Closing the gaps the framework exposes, in Soar and beyond, is our ongoing work.

Acknowledgments

We thank the anonymous reviewers for their insightful, in-depth, and helpful comments. Claude (Anthropic) was used to assist with drafting and editing the manuscript and with preparing figures. The authors developed the framework and analysis and are responsible for all content.

References

AlSayyad, A., Huang, K. Y., & Pal, R. (2026). AgentTrace: A structured logging framework for agent system observability. AAAI 2026 Workshop on Large Language Model-based Multi-Agent Systems (LaMAS).

Bovens, M. (2007). Analysing and assessing accountability: A conceptual framework. *European Law Journal*, *13*, 447–468.

Castelfranchi, C., & Falcone, R. (2010). *Trust theory: A socio-cognitive and computational model*. John Wiley & Sons.

Díaz-Rodríguez, N., Del Ser, J., Coeckelbergh, M., López de Prado, M., Herrera-Viedma, E., & Herrera, F. (2023). Connecting the dots in trustworthy artificial intelligence: From AI principles, ethics, and key requirements to responsible AI systems and regulation. *Information Fusion*, *99*.

European Union (2024). Regulation (EU) 2024/1689 of the European Parliament and of the Council (Artificial Intelligence Act). Official Journal of the European Union, L 2024/1689.

Frasca, T., Krause, E., Thielstrom, R., & Scheutz, M. (2022). A framework for robot self-assessment of expected task performance. *IEEE Robotics and Automation Letters*, *7*.

High-Level Expert Group on Artificial Intelligence (2019). *Ethics guidelines for trustworthy AI*. Technical report, European Commission, Brussels. From https://digital-strategy.ec.europa.eu/en/library/ethics-guidelines-trustworthy-ai.

Kadavath, S., Conerly, T., Askell, A., Henighan, T., Drain, D., Perez, E., et al. (2022). Language models (mostly) know what they know. *arXiv preprint arXiv:2207.05221*.

Kirk, J. R., Wray, R. E., Lindes, P., & Laird, J. E. (2024). Improving Knowledge Extraction from LLMs for Task Learning through Agent Analysis. *Proceedings of the 38th Annual AAAI Conference on Artificial Intelligence*. Vancouver: arXiv. ArXiv:2306.06770 [cs].

Kolve, E., et al. (2017). AI2-THOR: An interactive 3D environment for visual AI. ArXiv:1712.05474 [cs.CV].

Kuipers, B. (2018). How Can We Trust a Robot? *Communications of the ACM*, *61*, 86–95.

Laird, J. E. (2012). *The Soar cognitive architecture*. Cambridge, MA: MIT Press.

Laird, J. E., Lebiere, C., & Rosenbloom, P. S. (2017). A standard model of the mind: Toward a common computational framework across artificial intelligence, cognitive science, neuroscience, and robotics. *AI Magazine*, *38*, 13–26.

Lee, J. D., & See, K. A. (2004). Trust in automation: Designing for appropriate reliance. *Human Factors*, *46*, 50–80.

Liu, Y., et al. (2023). Trustworthy LLMs: A survey and guideline for evaluating large language models' alignment. ArXiv:2308.05374.

National Institute of Standards and Technology (2023). *Artificial intelligence risk management framework*. Technical Report NIST AI 100-1, National Institute of Standards and Technology.

Oruganti, S., Nirenburg, S., McShane, M., English, J., Roberts, M., Arndt, C., Parasuraman, R., & Sentis, L. (2026). Why cognitive robotics matters: Lessons from OntoAgent and LLM deployment in HARMONIC for safety-critical robot teaming. ArXiv:2603.26730 [cs.RO].

Qi, J., et al. (2026). Towards trustworthy agentic AI: A comprehensive survey of safety, robustness, privacy, and system security. *Academia AI and Applications*, *2*.

Stavropoulos, A. (2026). Commentary: Qi, J. et al. Towards trustworthy agentic AI: A comprehensive survey of safety, robustness, privacy, and system security. *Academia AI and Applications*, *2*.

Turpin, M., Michael, J., Perez, E., & Bowman, S. R. (2023). Language models don't always say what they think: Unfaithful explanations in chain-of-thought prompting. *Advances in Neural Information Processing Systems*.